Projet 1 – Clone d'`egrep`

Recherche de motif par automate et par KMP

Ramy Sail Younes Bensallah

UE DAAR – Master 2 – octobre 2026

1 Introduction

Le problème est celui de la commande `egrep` : étant donnés un motif R (une expression régulière) et un fichier texte de l lignes, afficher les lignes qui contiennent au moins un facteur appartenant au langage de R. Comme le propose le sujet, nous traitons chaque ligne indépendamment, et nous nous limitons au sous-ensemble suivant de la norme ERE : les lettres, le point (caractère universel), la concaténation, l'alternative `|`, l'étoile `*` et les parenthèses. Nous avons ajouté `+` en plus. Les autres opérateurs (`?`, `{}`, `[]`, `^`, `$`) sont lus comme des lettres.

Le programme suit les cinq étapes vues en cours :

1. motif → arbre syntaxique (`RegexParser.java`) ;
2. arbre → automate non déterministe avec ε-transitions (`Ndfa.java`), méthode d'Aho-Ullman ;
3. ε-AFN → automate déterministe, méthode des sous-ensembles (`Determinisation.java`) ;
4. automate déterministe → automate minimal, algorithme de Moore (`Minimisation.java`) ;
5. parcours de chaque ligne par l'automate (`Dfa.java`).

Lorsque l'arbre obtenu à l'étape 1 n'est qu'une suite de concaténations de lettres, le motif est un simple mot : les étapes 2 à 5 sont alors remplacées par l'algorithme de Knuth-Morris-Pratt (`Kmp.java`). Le choix est fait par la méthode `pour` de `Recherche.java`, et l'option `--dfa` force l'automate pour pouvoir comparer les deux méthodes. Sans option, la sortie est celle d'`egrep` (les lignes trouvées, telles quelles) et le code de retour aussi : 0 si une ligne est trouvée, 1 sinon, 2 en cas d'erreur. L'option `--detail` numérote les lignes et colore les occurrences.

Le code est en Java 21, sans dépendance. Les sections 2 à 7 décrivent chaque algorithme, la section 8 la manière dont nous avons vérifié les résultats, et la section 9 les performances, comparées à celles de GNU `grep`.

2 Du motif à l'arbre syntaxique

Structure de données. L'arbre (`RegexTree.java`) a cinq sortes de nœuds : `Feuille` (une lettre), `Point`, `Concat`, `Union` et `Etoile`. Chaque sorte est un `record` Java qui porte exactement ses fils, ce qui rend impossible un nœud `Union` à un seul fils, par exemple. Les parenthèses ne laissent pas de trace : elles ne font que modifier la forme de l'arbre.

Algorithme. En ERE la concaténation est implicite : dans `ab*`, rien ne sépare `a` de `b*`. Nous commençons donc par insérer un opérateur de concaténation explicite (un caractère de contrôle, qui ne peut pas apparaître dans un motif saisi) entre deux éléments qui se suivent. Le motif est ensuite mis en notation postfixe par l'algorithme *shunting-yard*, avec les priorités `*` = `+` > concaténation > `|`. Enfin, une pile reconstruit l'arbre à partir de la notation postfixe. Les deux passes sont linéaires en la longueur m du motif. Un motif mal formé (parenthèse non fermée, opérateur sans opérande) produit une erreur et le code de retour 2.

Remarques. `e+` est réécrit en `ee*`, ce qui duplique le sous-arbre e. C'est le plus simple, mais l'automate de l'étape 2 grossit d'autant : pour `(abc)+`, les états de `abc` sont créés deux fois. Une construction dédiée à `+` (une seule copie et une ε-transition de retour) éviterait ce doublement. Par ailleurs, nous refusons le motif vide, `()` et `a||b`, qu'`egrep` accepte ; ce sont des cas sans intérêt pratique, mais c'est un écart à la norme.

3 De l'arbre à l'automate non déterministe

Algorithme. Nous suivons la construction récursive du livre d'Aho et Ullman [1] (chapitre 10), connue aussi sous le nom de construction de Thompson [9]. Chaque sous-arbre produit un fragment d'automate qui a un seul état d'entrée et un seul état de sortie, et le nœud parent recolle les fragments de ses fils :

- une lettre c (ou le point) donne deux états reliés par une transition étiquetée c (ou par une transition « joker ») ;
- $e_1 e_2$ relie la sortie de e_1 à l'entrée de e_2 par une ε-transition, sans nouvel état ;
- $e_1 | e_2$ ajoute une entrée qui mène aux deux fragments et une sortie que les deux fragments rejoignent ;
- e^* ajoute une entrée et une sortie, une ε-transition de l'entrée vers la sortie (le mot vide) et une de la sortie de e vers son entrée (la répétition).

Structure de données et complexité. Les états sont des entiers ; pour chacun, l'automate garde la liste de ses transitions étiquetées et la liste de ses ε-transitions. Chaque feuille et chaque opérateur `|` ou `*` crée deux états, la concaténation aucun : sans `+`, l'automate a donc au plus $2m$ états. Chaque état a au plus une transition étiquetée ou deux ε-transitions sortantes, si bien que le nombre de transitions est aussi linéaire. La construction se fait en $O(m)$.

Remarque. Cet automate est petit mais coûteux à simuler : à chaque caractère lu, il faut suivre tout un ensemble d'états et calculer sa ε-fermeture, soit $O(m)$ opérations par caractère. C'est ce qui justifie les étapes 3 et 4.

4 Déterminisation par la méthode des sous-ensembles

Algorithme. La méthode des sous-ensembles (Rabin et Scott [8]) construit un automate déterministe (DFA) dont chaque état représente un ensemble d'états de l'ε-AFN. L'état initial est l'ε-fermeture de l'état initial de l'ε-AFN. Pour un état E et un symbole c, la transition mène à l'ε-fermeture des états atteints depuis E en lisant c. Un état est final s'il contient l'état final de l'ε-AFN. On part de l'état initial et on ne crée que les états accessibles, à l'aide d'une file et d'une table de hachage qui associe à chaque ensemble déjà rencontré son numéro dans le DFA.

Un alphabet réduit. Le texte est lu en caractères Java (UTF-16) : un DFA complet aurait 65 536 colonnes par état, presque toutes identiques. Or tous les caractères qui n'apparaissent pas dans le motif se comportent de la même façon : seules les transitions joker les acceptent. Nous gardons donc une colonne par lettre du motif, plus une seule colonne « autre » pour tous les autres caractères. L'automate est en outre partiel : l'absence de transition tient lieu d'état puits. C'est ce qui rend le point utilisable sans énumérer l'alphabet.

Structure de données. Le DFA range ses transitions dans un tableau `int[][]` : la case $[q][j]$ donne l'état atteint depuis q par la colonne j, ou -1 s'il n'y a pas de transition. Un second tableau, indexé par le code du caractère et de la taille du plus grand caractère du motif plus un, donne la colonne de chaque caractère ; un caractère absent du motif, ou au-delà du tableau, va

dans la colonne « autre ». Lire un caractère coûte ainsi deux accès à un tableau, sans allocation. Notre première version utilisait une `HashMap` par état, nettement plus lente (section 9.4).

Complexité. Pour un ε-AFN à n états, le DFA peut avoir jusqu'à 2^n états, et chacun coûte $O(n)$ par symbole à construire. Ce pire cas est atteint : la section 9.5 le mesure sur une famille de motifs dont le DFA a 2^{k+1} états. Sur des motifs courants en revanche, l'explosion ne se produit pas : pour les huit motifs de la section 9, la construction complète (étapes 1 à 4) prend entre 0,06 et 0,45 ms.

5 Minimisation

Algorithme. Nous utilisons l'algorithme de Moore [7], par raffinement de partition. On part de deux classes, les états finaux et les autres. À chaque tour, deux états restent dans la même classe s'ils y étaient déjà et si, pour chaque symbole, leurs successeurs sont dans la même classe. Concrètement, chaque état reçoit une « signature » (sa classe, puis la classe de son successeur pour chaque colonne), et les états de même signature forment la nouvelle classe. On s'arrête quand le nombre de classes ne change plus. Comme notre DFA est partiel, nous ajoutons l'état puits pendant le calcul, puis nous retirons sa classe, qui contient aussi les éventuels états morts. L'automate minimal est reconstruit avec un représentant par classe.

Complexité et littérature. Chaque tour coûte $O(n\,|\Sigma|)$ et il y a au plus n tours, d'où $O(n^2|\Sigma|)$ dans le pire cas. L'algorithme de Hopcroft [5] atteint $O(|\Sigma|\,n \log n)$ en ne raffinant que par rapport à la plus petite des deux moitiés d'une classe coupée. Nous avons gardé Moore, plus simple, car la minimisation ne coûte presque rien ici : sur nos motifs, les étapes 1 à 4 réunies prennent moins d'une demi-milliseconde.

Apport réel. La minimisation est garantie optimale, mais elle gagne peu sur nos motifs. Pour `S(a|g|r)+on`, l'automate minimal a cinq états (figure 1), et sur la famille de la section 9.5 elle ne retire qu'un état sur $2^{k+1}+1$. Elle reste utile quand le motif contient des redondances, par exemple `a|a` ou `(a*b*)*`, et elle nous a servi de vérification : un automate minimal est unique, ce qui donne un point de comparaison fiable dans les tests.

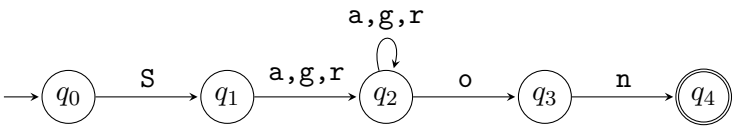


Figure 1 – Automate minimal de `S(a|g|r)+on`, produit par le programme. Les transitions absentes mènent à l'état puits, qui n'est pas représenté.

6 Recherche dans une ligne

Le problème. L'automate reconnaît les mots du langage de R, alors qu'il faut savoir si une ligne en *contient* un. La méthode vue en cours lance l'automate depuis chaque position de la ligne, ce qui coûte $O(n^2)$ pour une ligne de n caractères dans le pire cas.

Notre choix. Nous construisons l'automate de `.*`R plutôt que celui de R. Une ligne contient un facteur reconnu par R si et seulement si l'un de ses préfixes est reconnu par `.*`R. Il suffit donc d'un seul parcours de la ligne, arrêté au premier état final rencontré : la recherche devient linéaire, $O(n)$ par ligne. L'arbre de `.*`R est construit directement, sans repasser par le texte du motif, et suit ensuite les étapes 2 à 4 comme les autres.

Le prix à payer. L'automate de `.*`R peut être beaucoup plus gros que celui de R, car il doit se souvenir de toutes les occurrences commencées en même temps. Pour $R = \texttt{a(a|b)}^k$, l'automate minimal de R est une simple chaîne de $k+2$ états, alors que celui de `.*`R en a 2^{k+1} : il doit retenir lesquels des $k+1$ derniers caractères étaient des `a`. C'est la famille mesurée à la section 9.5. Pour les motifs d'un utilisateur d'`egrep`, ce cas est rare, et le gain linéaire l'emporte.

Repérage des occurrences. L'option `--detail` doit en plus colorer chaque occurrence, ce qui demande ses bornes. Nous utilisons alors un second automate, celui de R seul, que nous lançons à chaque position jusqu'à trouver une occurrence, en gardant la plus longue, puis nous reprenons après sa fin, comme `egrep --color`. Ce calcul est quadratique dans le pire cas, mais il n'est fait que sur les lignes déjà retenues, et seulement avec `--detail`.

7 Le cas d'un mot : Knuth-Morris-Pratt

Problème et algorithme. Quand le motif est un mot F de longueur m, la recherche revient à trouver un facteur dans une chaîne. L'algorithme naïf compare F à chaque position et recule dans le texte après un échec, d'où $O(nm)$. L'algorithme de Knuth, Morris et Pratt [6] ne recule jamais dans le texte : quand le caractère $F[j]$ ne correspond pas, une table de report indique à quel indice de F reprendre la comparaison. $\mathit{report}[j]$ est la longueur du plus long bord propre de $F[0..j-1]$ (un préfixe qui est aussi un suffixe), et vaut -1 quand il faut avancer d'un caractère dans le texte. Comme dans le cours, nous utilisons la version améliorée de la table : si $F[\mathit{report}[j]] = F[j]$, reprendre à $\mathit{report}[j]$ échouerait sur le même caractère, on prend donc directement $\mathit{report}[\mathit{report}[j]]$. Nos tables sont celles du cours, par exemple $[-1, 0, -1, 1, 0]$ pour `mami` et $[-1, 0, 0, -1, 0, 2, 0]$ pour `chicha`.

Complexité. Le calcul de la table coûte $O(m)$ et la recherche $O(n)$: à chaque caractère du texte, l'indice j augmente au plus de un, et chaque report le fait diminuer, donc le nombre total de reports est borné par n. La structure de données se réduit au tableau des caractères de F et au tableau *report*.

Critique. KMP a la même complexité que notre automate de `.*`R (qui est d'ailleurs, pour un mot, essentiellement l'automate des occurrences de F), et il n'a rien à construire. En pratique, il est 1,1 à 1,7 fois plus rapide que notre automate (section 9) : pour chaque caractère du texte, il compare deux caractères, alors que l'automate lit deux tableaux. Avec notre première table de transitions, à base de `HashMap`, l'écart était d'un facteur trois environ (section 9.4). Il n'est pas pour autant optimal : l'algorithme de Boyer-Moore [2] compare le mot en partant de sa fin et peut sauter jusqu'à m caractères d'un coup. Il est sous-linéaire en moyenne sur un texte en langue naturelle, et c'est lui qu'utilise GNU `grep` pour les mots [4]. L'avantage de KMP est de lire chaque caractère une seule fois et dans l'ordre, ce qui convient à un flux qu'on ne peut pas relire.

8 Tests

Instances. Comme le recommande le sujet, nos textes viennent du projet Gutenberg : six livres de tailles et de styles différents, dont le texte de la figure 1 du sujet (*A History of Babylon*, 763 Ko), *Alice*, *Sherlock Holmes*, *Moby Dick*, *Guerre et Paix* (3,4 Mo) et *Candide*, en français, pour les caractères accentués. Ils sont gardés tels que Gutenberg les distribue, en UTF-8 avec des fins de ligne CRLF. Le corpus fait 6,4 Mo et 121 817 lignes. Pour les mesures selon la taille, il est recopié jusqu'à 32 Mo.

Tests unitaires. 60 tests JUnit couvrent chaque étape. Les plus utiles comparent deux implémentations indépendantes sur *toutes* les entrées courtes : l'ε-AFN simulé directement, le DFA et le DFA minimal doivent accepter exactement les mêmes mots parmi les 1 365 mots de longueur au plus 5 sur l'alphabet $\{a, b, c, x\}$ (x n'apparaît dans aucun motif et teste la colonne « autre ») ; KMP doit donner le même résultat que `String.contains`, puis que l'automate, sur tous les mots de longueur au plus 8 sur $\{a, b\}$. D'autres tests vérifient les tables de report du cours, les cinq états de l'automate minimal de `S(a|g|r)+on`, et le fait que minimiser un automate déjà minimal ne change rien.

Comparaison avec `egrep`. Le sujet demande le même résultat qu'`egrep`. Notre banc d'essai le vérifie à chaque exécution : pour chacun des huit motifs de la section 9, la sortie du clone est comparée octet par octet à celle de GNU `grep -E` sur tout le corpus, et KMP et l'automate doivent trouver les mêmes lignes. Un écart avec `egrep` est apparu en cours de projet, sur les fins de ligne : nous lisions d'abord le fichier avec `BufferedReader.readLine`, qui coupe aussi sur `\r`. Or `egrep` ne coupe que sur `\n` et garde le `\r` dans la ligne, où le point peut le reconnaître. Nous coupons désormais le fichier sur l'octet `\n`, qui ne peut pas apparaître au milieu d'un caractère UTF-8, avant de décoder chaque ligne. Dans les conditions de mesure (`LC_ALL=C.UTF-8`), les sorties sont identiques pour les huit motifs.

9 Performances

Protocole. Les mesures ont été faites sur un Intel Core i7-1360P, sous Linux (WSL2), avec Java 21 et GNU `grep` 3.11. Chaque mesure est répétée 10 fois après 3 exécutions à vide, qui laissent le compilateur à la volée de Java optimiser le code ; nous donnons la moyenne et l'écart type, représenté par les barres d'erreur. Nous mesurons à deux niveaux :

- *recherche seule* : l'appel de la recherche sur chaque ligne, les lignes étant déjà en mémoire. C'est le coût de l'algorithme ;
- *commande complète* : `java -jar egrep.jar` ou `egrep`, lancés comme dans un terminal. C'est ce que voit l'utilisateur, et c'est la seule façon de comparer le clone à `egrep`.

La sortie des commandes est écrite dans un fichier et non dans `/dev/null` : GNU `grep` détecte `/dev/null` et s'arrête à la première ligne trouvée, ce qui faussait nos premières mesures (1 ms au lieu de 18).

9.1 Selon la taille du texte

La figure 2 montre que tous les temps sont linéaires en la taille du texte, ce qu'annonce l'analyse. Pour la recherche seule, KMP traite environ 1,8 ms par Mo et l'automate 2,7 ms par Mo sur le mot `Babylon`. Le motif `S(a|g|r)+on` coûte à peu près autant que `Babylon` avec l'automate (12 % de plus à 32 Mo), car le DFA fait une transition par caractère quel que soit le motif : une fois l'automate construit, la complexité du motif ne compte plus.

L'automate reste un peu plus lent que KMP, alors que les deux lisent chaque caractère une fois : il fait deux accès à un tableau par caractère, contre une comparaison de deux caractères pour KMP, qui reporte rarement sur un texte en langue naturelle. L'écart était bien plus grand avec notre première table de transitions (section 9.4).

9.2 Selon le motif

La figure 3 donne le temps de huit motifs sur tout le corpus : trois mots plus ou moins fréquents, puis un exemple pour chaque opérateur. Pour la recherche seule avec l'automate, les motifs fréquents `the` et `th(e|is|at)` sont les plus rapides : ils apparaissent dans environ la moitié des lignes, et la recherche s'arrête à la première occurrence au lieu de lire la ligne entière. Les

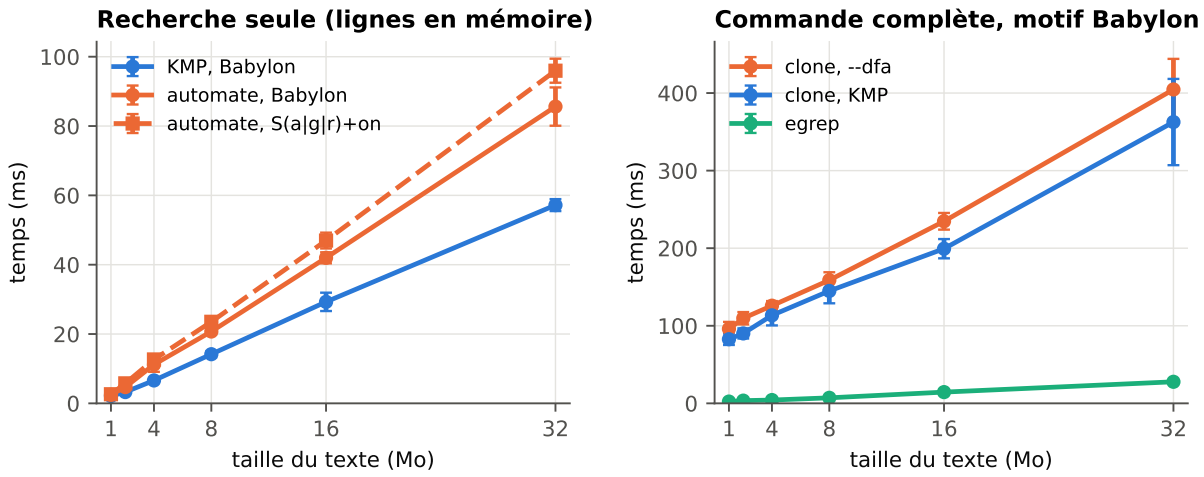


Figure 2 – Temps de recherche selon la taille du texte (corpus recopié de 1 à 32 Mo). À gauche, la recherche seule ; à droite, la commande complète, démarrage de Java compris.

motifs rares, comme `Babylon` ou `colou*r`, obligent au contraire à lire presque toutes les lignes jusqu'au bout. Sur les trois mots, KMP est 1,1 à 1,7 fois plus rapide que l'automate.

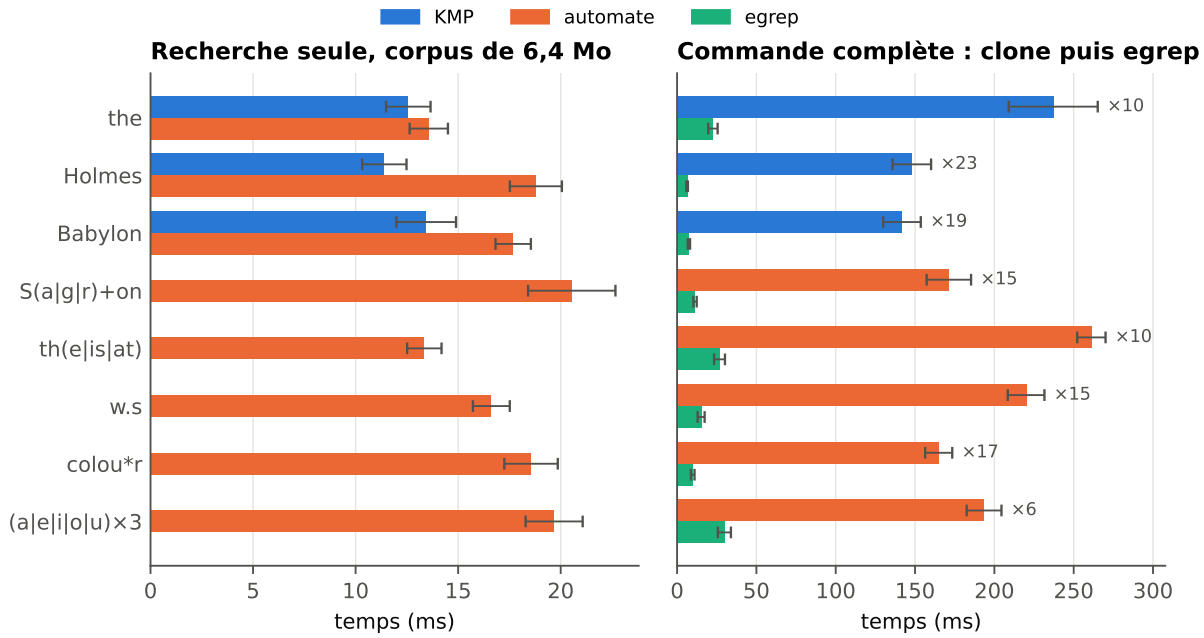


Figure 3 – Temps par motif sur le corpus complet (6,4 Mo). À gauche, la recherche seule ; pour les trois mots, KMP et l'automate. À droite, la commande complète du clone (avec la méthode qu'il choisit) et celle d'`egrep` ; le facteur indique combien de fois le clone est plus lent.

9.3 Comparaison avec `egrep`

En commande complète, le clone est 6 à 23 fois plus lent qu'`egrep` (figure 3, à droite). Une partie de l'écart est fixe : sur un fichier vide, le clone met 55 ms (écart type 4,1) à s'exécuter, contre 1,4 ms pour `egrep`, et c'est presque entièrement le démarrage de la machine virtuelle Java. Le reste grandit avec le texte : à 32 Mo, la commande complète avec KMP prend 363 ms alors que la recherche seule n'en prend que 57. La différence correspond à ce que le clone fait en plus de chercher : lire le fichier entier, le découper, décoder chaque ligne en chaîne Java, écrire les lignes

trouvées, le tout au début avec du code que le compilateur à la volée n'a pas encore optimisé. Le nombre de lignes écrites compte aussi : avec KMP, `the` et `Holmes` ont presque le même temps de recherche (13 et 11 ms), mais la commande complète coûte près de 90 ms de plus pour `the`, qui fait écrire 56 018 lignes au lieu de 460.

GNU `grep` évite presque tous ces coûts [4] : il travaille sur les octets sans les décoder, lit le fichier par blocs, cherche le motif dans le bloc entier et ne délimite la ligne qu'autour d'une occurrence trouvée. Pour un mot, il utilise Boyer-Moore, qui ne regarde qu'une partie des caractères ; pour une expression régulière, il construit l'automate déterministe à la demande, seulement pour les états réellement atteints. Enfin, les commandes complètes sont nettement plus dispersées que la recherche seule (écart type jusqu'à 56 ms pour `Babylon` à 32 Mo avec KMP), ce que nous attribuons au système de fichiers de WSL2 et à la variabilité du démarrage de Java.

9.4 Table de transitions : d'une `HashMap` à un tableau

Notre première version du DFA rangeait les transitions de chaque état dans une table de hachage (`HashMap`), qui associait à chaque `Character` un `Integer`. Chaque caractère lu coûtait alors un accès à la liste des états, la conversion du `char` en objet `Character` (une allocation au-delà du code 127, par exemple pour les lettres accentuées), un calcul de hachage, la recherche dans la table, puis la conversion de l'`Integer` obtenu. La version actuelle (section 4) remplace tout cela par deux accès à un tableau. Seul le fichier `Dfa.java` a changé entre les deux séries de mesures.

Gain. En recherche seule, l'automate est 3,4 à 5,6 fois plus rapide selon le motif (figure 4) : il passe de 12,4 à 2,7 ms par Mo sur `Babylon`. Le gain est plus faible en commande complète, où le démarrage de Java, la lecture et le découpage du fichier ne changent pas : à 32 Mo, la commande avec `--dfa` passe de 645 à 405 ms, et les cinq motifs de la figure 3 qui utilisent l'automate gagnent 28 à 46 %. Le tableau est dense : $n(|\Sigma| + 1)$ entiers pour n états, là où la `HashMap` ne stocke que les transitions présentes, mais au prix de plusieurs dizaines d'octets par transition. Même pour les 8 192 états de la section 9.5, le tableau occupe environ 300 Ko.

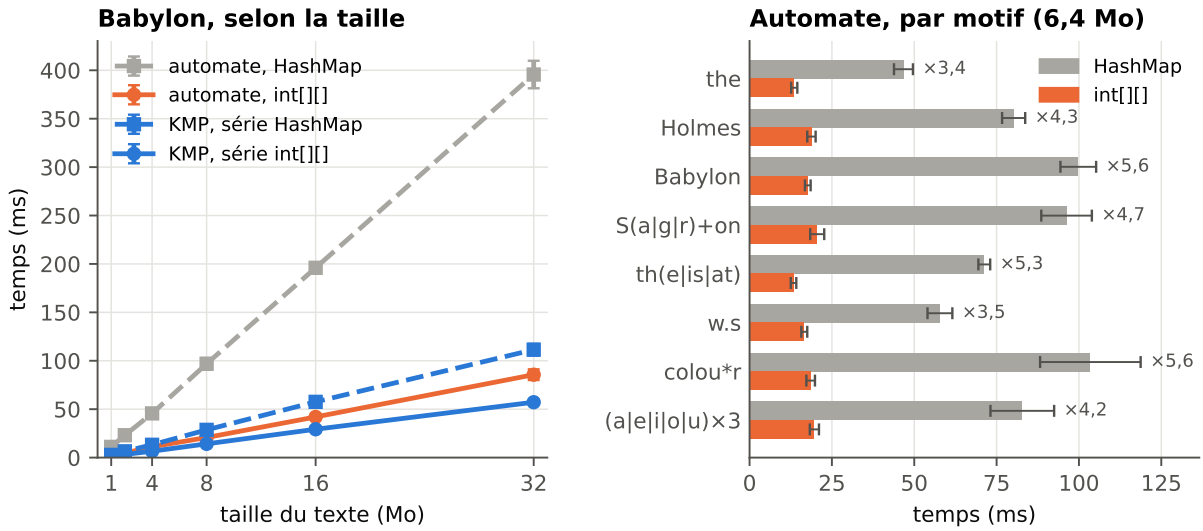


Figure 4 – Recherche seule avec la première table de transitions (`HashMap`, en gris) puis avec le tableau `int[][]`. À gauche, `Babylon` selon la taille du texte, avec KMP mesuré dans chacune des deux séries ; à droite, l'automate sur chaque motif du corpus complet, avec le facteur gagné.

9.5 Explosion du nombre d'états

La figure 5 mesure le pire cas de la méthode des sous-ensembles, sur `a(a|b)`^{k} recherché dans une ligne (donc l'automate de `.*a(a|b)`^{k}), pour k de 1 à 12. L'ε-AFN grandit linéairement ($6k + 6$ états, soit 78 pour $k = 12$), alors que le DFA en a $2^{k+1} + 1$ et le DFA minimal 2^{k+1} : la minimisation ne peut rien fusionner, puisque ces états correspondent à des contenus différents des $k + 1$ derniers caractères. Le temps de construction suit, et double à peu près à chaque k : 92 ms pour 8 192 états. C'est le risque de la déterminisation complète, et la raison pour laquelle les outils réels la font à la demande.

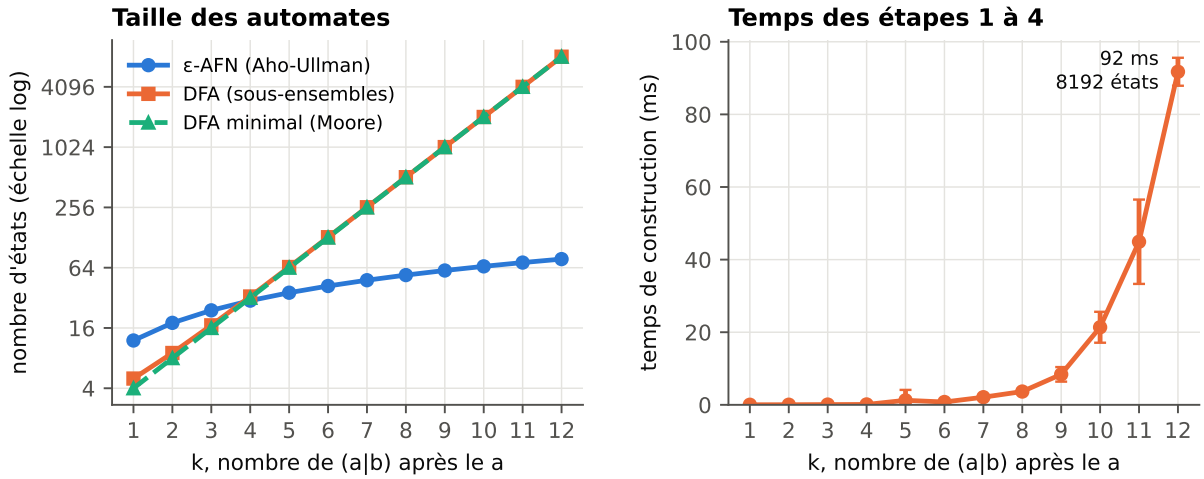


Figure 5 – Motif `a(a|b)`^{k} : nombre d'états des trois automates (échelle logarithmique) et temps de construction des étapes 1 à 4.

10 Conclusion et perspectives

Notre clone implémente les cinq étapes du cours et KMP, et il donne exactement la sortie d'`egrep` sur nos instances. La recherche est linéaire grâce à l'automate de `.*`R, au prix d'un automate qui peut devenir exponentiel sur des motifs construits pour cela. Les mesures montrent que la complexité asymptotique ne suffit pas à expliquer les temps : à complexité égale, remplacer une table de hachage par un tableau a rendu l'automate 3 à 6 fois plus rapide, assez pour le rapprocher de KMP, et le clone reste 6 à 23 fois plus lent qu'`egrep`, surtout à cause de ce qui entoure la recherche.

Plusieurs améliorations sont possibles, dans l'ordre de leur rapport gain/effort :

— **chercher sur les octets** plutôt que sur des chaînes décodées, et ne délimiter les lignes qu'autour des occurrences, comme `grep` ;

— **Boyer-Moore** à la place de KMP pour les mots, et, pour une expression régulière, chercher d'abord avec Boyer-Moore un mot que toute occurrence doit contenir (`on` dans `S(a|g|r)+on`) avant de lancer l'automate ;

— **une déterminisation à la demande**, qui ne crée un état du DFA que lorsqu'il est atteint par le texte et supprime l'explosion de la section 9.5 dans la pratique [3].

Enfin, si l'on cherche souvent dans les mêmes textes, comme un moteur de recherche hors ligne, il vaut mieux payer une fois un précalcul : une table d'index de tous les mots du texte, consultée par une table de hachage, un arbre de préfixes (*P-trie*) ou un filtre de Bloom. La recherche d'un mot ne dépend alors plus de la taille du texte. Pour une expression régulière, on peut lancer l'automate sur la liste des mots de l'index, bien plus courte que le texte, au lieu du texte entier.

Usage de l'intelligence artificielle

Dans le cadre de ce projet, nous avons utilisé Claude afin de générer les mesures ainsi que les scripts produisant les figures. Egalement pour l'implémentation de l'algorithme KMP, et pour le passage de la table de transitions du DFA à un tableau `int[][]`.

Références

[1] A. V. Aho, J. D. Ullman. *Foundations of Computer Science*, chapitre 10, « Patterns, Automata, and Regular Expressions ». Computer Science Press, 1992.

[2] R. S. Boyer, J. S. Moore. A fast string searching algorithm. *Communications of the ACM*, 20(10), p. 762–772, 1977.

[3] R. Cox. Regular expression matching can be simple and fast. `https://swtch.com/~rsc/regexp/regexp1.html`, 2007.

[4] M. Haertel. Why GNU grep is fast. Liste de diffusion `freebsd-current`, 2010.

[5] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. *Theory of Machines and Computations*, Academic Press, 1971.

[6] D. E. Knuth, J. H. Morris, V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), p. 323–350, 1977.

[7] E. F. Moore. Gedanken-experiments on sequential machines. *Automata Studies*, Princeton University Press, 1956.

[8] M. O. Rabin, D. Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2), p. 114–125, 1959.

[9] K. Thompson. Regular expression search algorithm. *Communications of the ACM*, 11(6), p. 419–422, 1968.